

STOCKPILOT LEARNING SERIES

StockPilot Frontend Handbook

A beginner-friendly guide to Next.js, React state, API integration, authentication, async polling, responsive UI, deployment, debugging, and interviews.

Use this document to

- Understand every screen and data flow
- Learn how the frontend talks to FastAPI
- Prepare practical frontend interview answers

Live: <https://stockpilot-analyzer.vercel.app>

How to use this handbook

Read the first ten sections to understand the implementation. Use the deployment checklist when publishing and the final question bank before interviews.

Goal

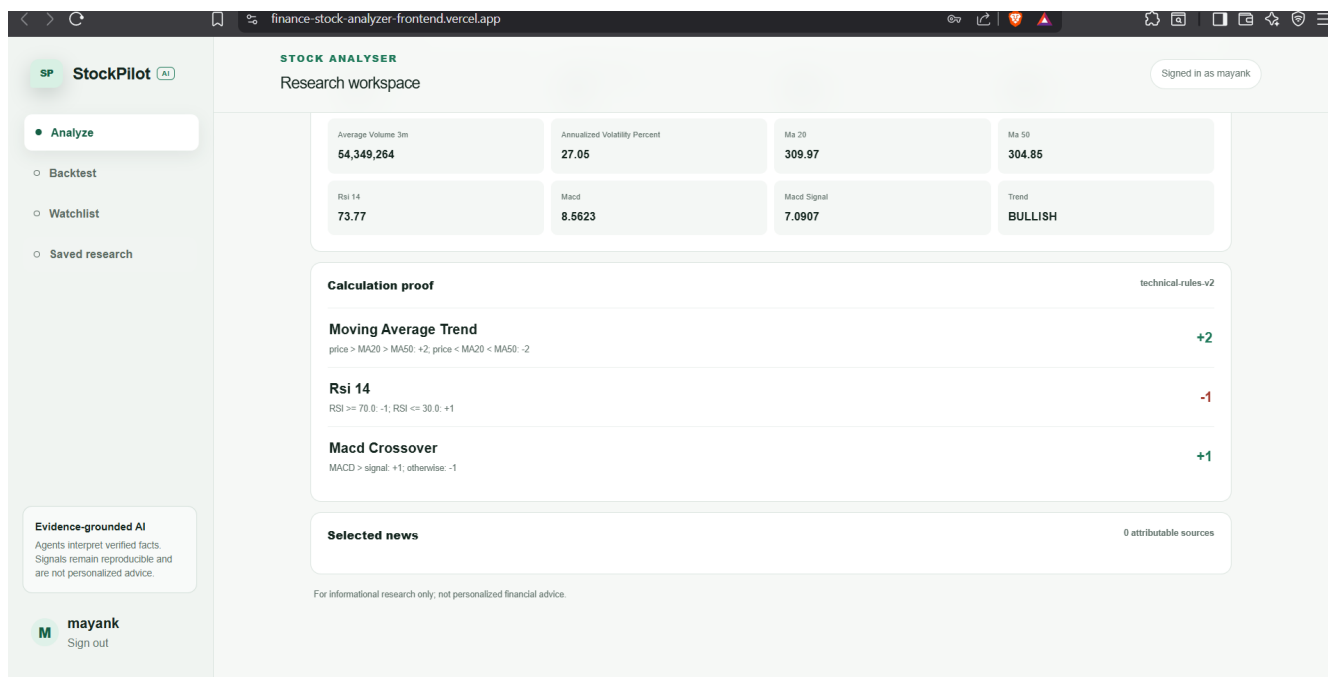
After reading this guide, you should be able to run the project, explain every major design decision, diagnose common failures, and answer interview questions without memorizing buzzwords.

Learning path

- 01 Frontend purpose and user journey
- 02 Architecture and project files
- 03 Next.js, React, and TypeScript basics
- 04 State model and rendering
- 05 Authentication flow
- 06 Asynchronous analysis polling
- 07 Backtesting, watchlist, and saved research
- 08 API client and error handling
- 09 Evidence-first UX
- 10 CSS design system, responsiveness, and accessibility
- 11 Environment variables and local setup
- 12 Vercel deployment and CORS
- 13 Testing and troubleshooting
- 14 Interview questions and model answers
- 15 Limitations and sensible improvements

1. What the frontend does

The frontend is the visual and interaction layer of StockPilot. It collects user input, calls the protected FastAPI endpoints, shows background progress, turns technical JSON into understandable cards, and lets each signed-in user manage personal research.



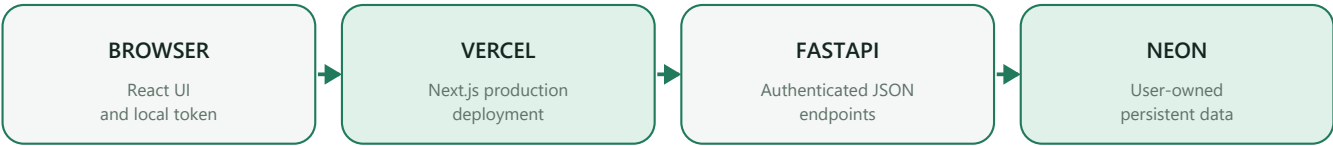
Production research workspace. The interface keeps headline decisions simple and places technical proof in expandable sections.

Four main workspaces

Workspace	User goal
Analyze	Research a ticker and inspect signal, evidence, proof, and news.
Backtest	Compare the rules-based strategy with buy-and-hold.
Watchlist	Keep frequently researched tickers one click away.
Saved research	Reopen a historical result exactly as it was saved.

2. Frontend architecture

The project is a separate Next.js application. Separation lets Vercel deploy the UI independently from the Render backend.



Gemini and database secrets never enter the browser.

File	Responsibility
app/page.tsx	Types, API helper, state, event handlers, and page rendering.
app/globals.css	Colors, layout, components, animation, responsive breakpoints.
app/layout.tsx	Metadata, global fonts, and root HTML structure.
package.json	Next.js commands, dependencies, linting, and build validation.
.env.example	Safe public API URL template.
next.config.ts	Next.js configuration.

Current shape

The interface is intentionally compact: one main client component manages this demo. In a larger product, state and UI sections would be split into hooks, services, and reusable components.

3. Next.js, React, and TypeScript basics

These technologies solve different parts of the frontend problem.

Technology	Simple explanation	StockPilot use
Next.js	A production framework built around React.	Routing, metadata, fonts, build, and Vercel deployment.
React	Builds interfaces from state-driven components.	Forms, tabs, loading states, and result rendering.
TypeScript	JavaScript with compile-time types.	Defines User, Job, Analysis, Backtest, News, and saved-data shapes.
CSS	Controls visual layout and interaction states.	Green design system, responsive grids, cards, and motion.

Why 'use client'?

The page uses browser-only behavior such as form events, localStorage, state hooks, effects, timers, and fetch. Marking it as a client component tells Next.js that this code must run interactively in the browser.

Why TypeScript types matter

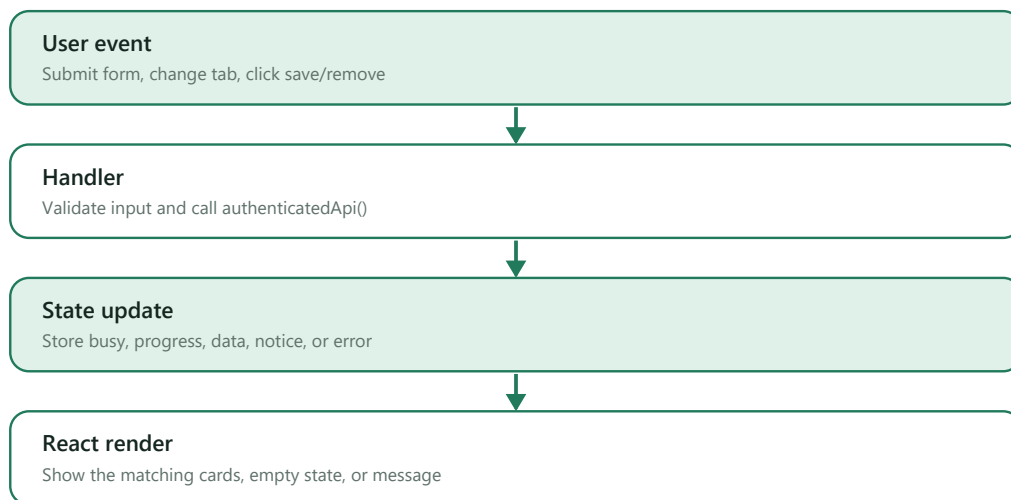
The Analysis and Backtest types document the expected API contract and help the editor catch mistakes such as reading a field that does not exist. Runtime responses can still fail, so the API helper also checks HTTP status and response bodies.

4. The frontend state model

React state is the page's memory. A state change triggers a re-render so the interface always reflects the latest job, result, or selected tab.

State	Meaning
token	JWT loaded from sessionStorage.
user	Current authenticated user.
authMode/authBusy/authError	Login/register form state.
tab	analyze, backtest, watchlist, or saved.
ticker	Normalized symbol currently being researched.
job	Queued/running/completed/failed analysis job.
analysis	Latest completed analysis response.
backtest	Latest backtest response.
watchlist/saved	Personal data loaded from the backend.
notice/errors	Visible feedback for user actions.

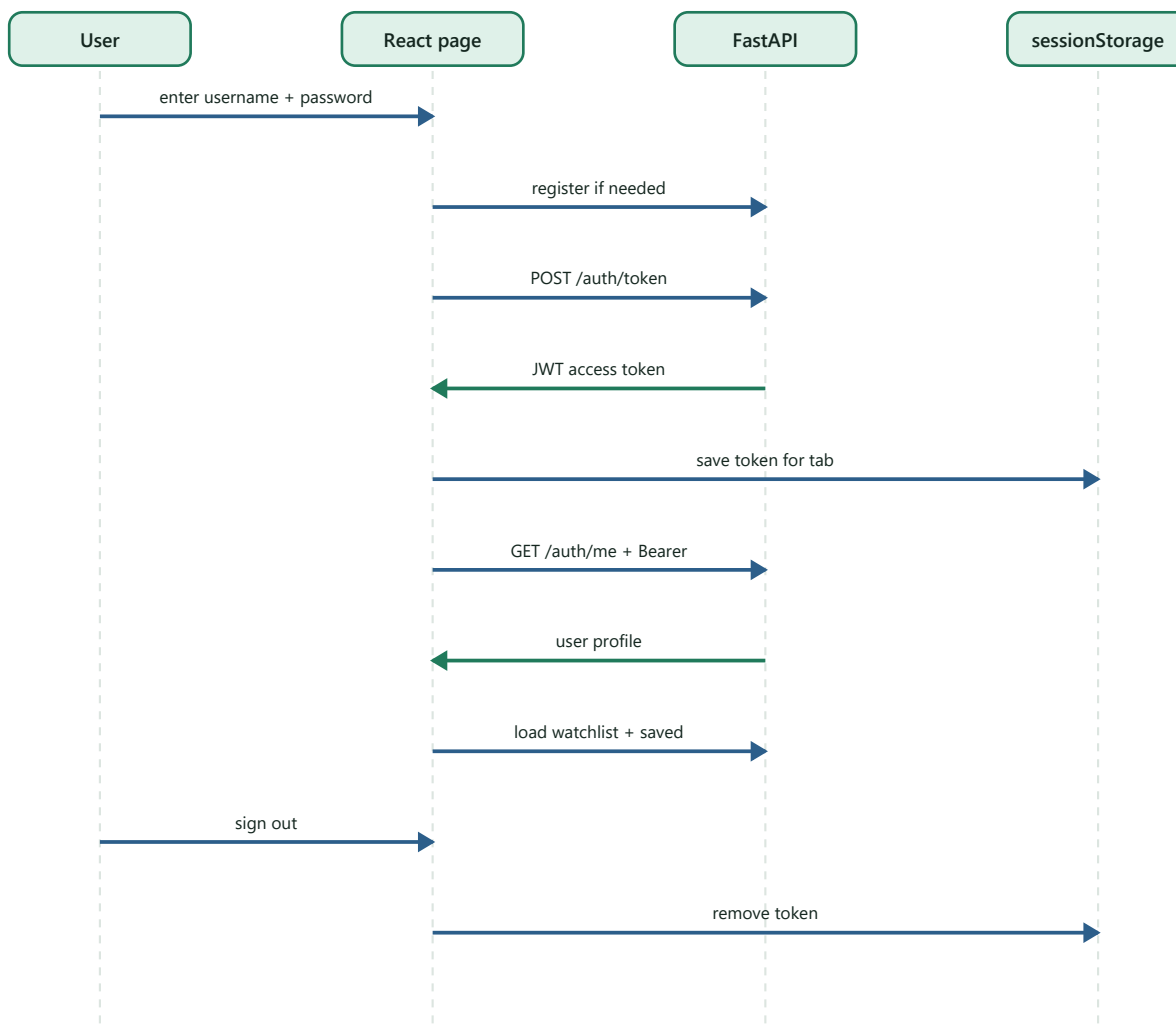
React follows event -> state -> UI.



useMemo creates the 'Signed in as ...' label from the user. useCallback keeps authenticated API and data-loading functions stable. useRef stores whether polling should continue without causing visual re-renders.

5. Authentication flow

The browser stores the access token for the current tab session and sends it with protected API calls.



Registration and login are separate operations. After registration, the UI requests a token using URL-encoded OAuth2 credentials. Protected calls add Authorization: Bearer <token>.

Security boundary

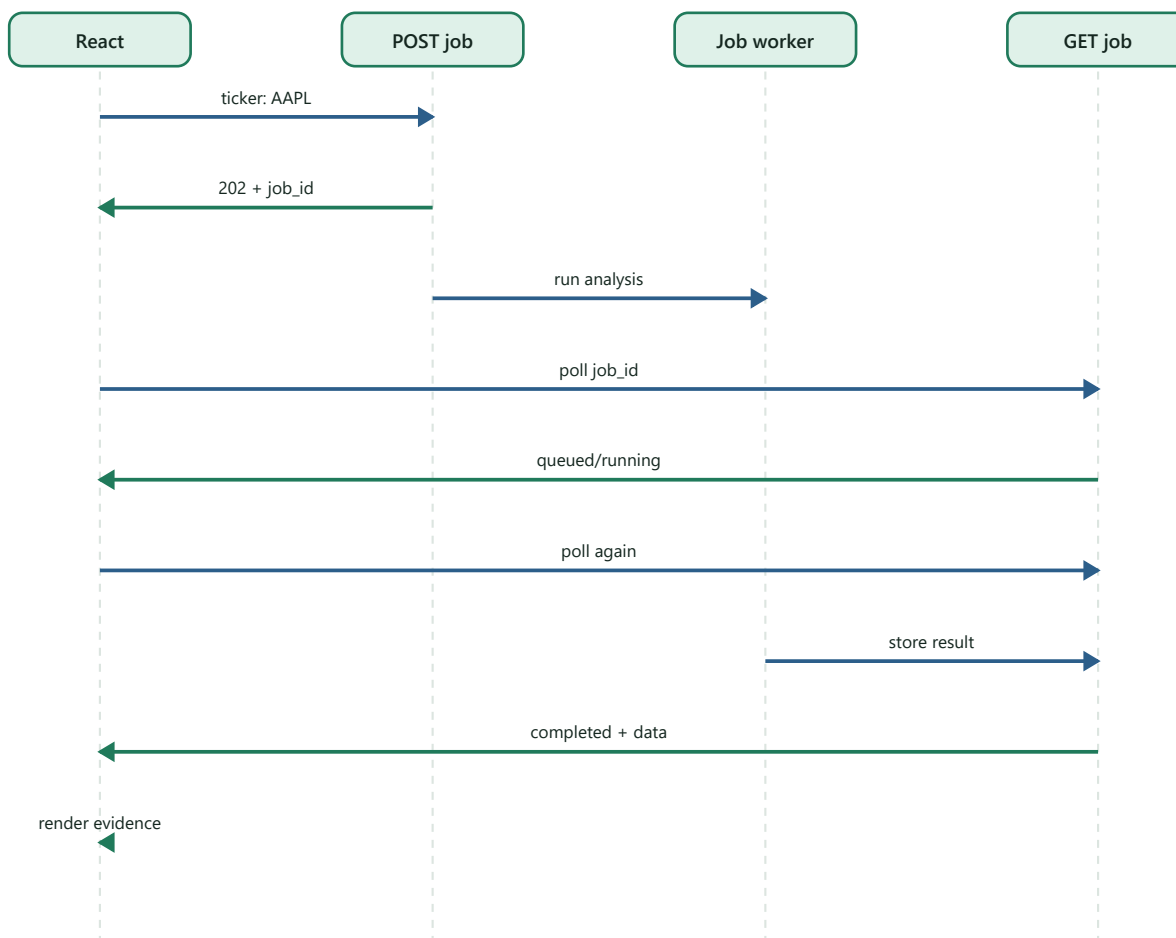
NEXT_PUBLIC_API_URL is safe to expose because it is only an address. Gemini keys, JWT signing secrets, and database credentials belong only on the backend host.

Why sessionStorage?

The token survives page refreshes but is removed when the browser tab session ends. This is convenient for a demo. A hardened application may prefer secure HTTP-only cookies with CSRF protection because JavaScript cannot read those cookies.

6. Asynchronous analysis and polling

Agent analysis can take longer than an ordinary API request. The UI starts a job and polls for progress instead of leaving one long request open.



`runAnalysis()` normalizes the ticker, clears previous errors, creates a job, and repeatedly requests its status. The progress card maps queued/running states to understandable steps. Completed data becomes analysis state; a failed job becomes a visible error.

Why polling?

Polling is simple, works through ordinary HTTP infrastructure, and is sufficient for a low-traffic demo. WebSockets or Server-Sent Events could provide push updates, but they add deployment and reconnection complexity.

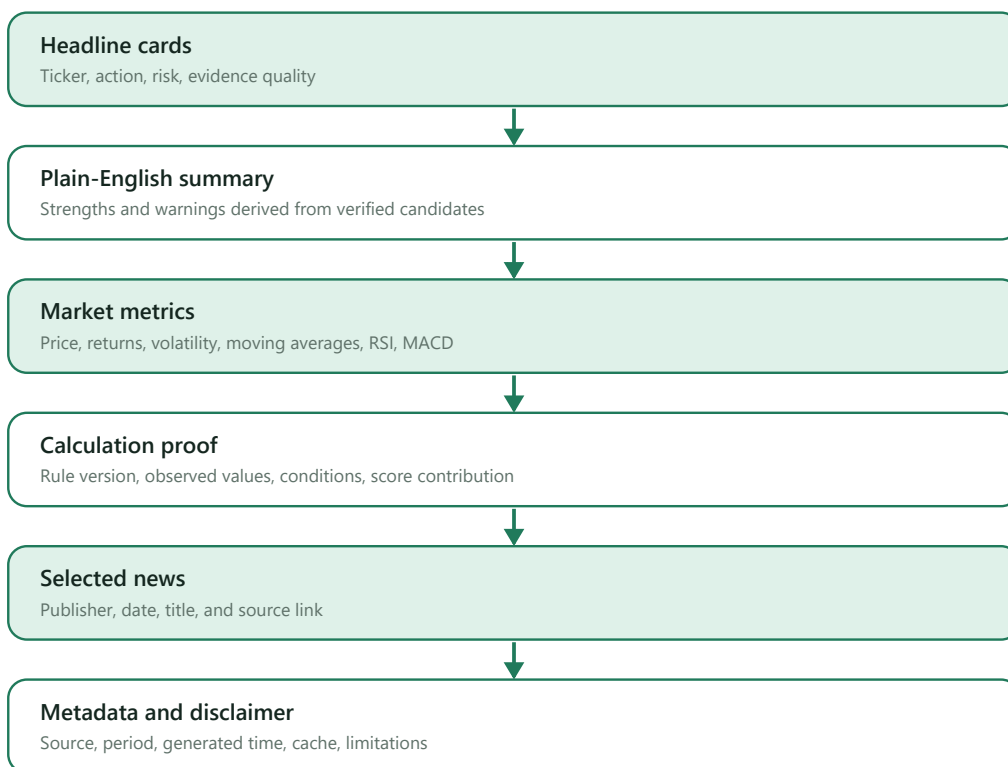
User experience principle

Never show an unexplained spinner for a long task. StockPilot shows that evidence collection and agent review are active, and it preserves a clear failure state.

7. Rendering analysis results

The result view uses progressive disclosure: show the decision first, then let the user open deeper evidence.

Important first; technical detail on demand.



Action colors are semantic: green for BUY, amber for HOLD, and red for SELL. The evidence-quality label explicitly avoids calling itself prediction confidence. Source links let the user verify claims outside the application.

Formatting helpers

Helper	Purpose
formatKey	Turns API snake_case labels into readable headings.
formatValue	Displays nulls, arrays, objects, and scalar values safely.
formatMoney	Formats monetary values consistently.
formatMetric	Applies readable units and formatting to technical metrics.

8. Backtest, watchlist, and saved research

These sections reuse the authenticated API helper but serve different user goals.

Backtest

The form sends ticker, period, initial capital, transaction costs, and optional parameters to POST /backtest. The UI begins with a plain-English verdict and strategy-versus-benchmark comparison, then exposes advanced metrics and calculation proof.

Watchlist

POST /watchlist adds a normalized ticker, GET /watchlist loads personal items, and DELETE /watchlist/{ticker} removes one. The Research button moves the ticker back into the analysis workflow. A friendly empty state explains how to add the first item.

Saved research

Saved analyses preserve the full JSON result from that point in time. Selecting a card restores it into analysis state without recomputing market data. Dates are formatted using the browser's locale and timezone.

Why saved data may appear empty

Local SQLite and deployed Neon are separate databases. A new production account starts with no personal records. Data appears after the signed-in user adds or saves it.

9. API client and error handling

A single `api()` helper gives every request consistent headers, response parsing, and error messages.

Request behavior

- Build the full URL from `NEXT_PUBLIC_API_URL` and the endpoint path.
- Set Content-Type: `application/json` when a JSON body is sent.
- Attach Authorization: Bearer token for `authenticatedApi()`.
- Treat HTTP 204 as a successful response without JSON.
- Parse JSON safely and throw a readable backend detail when the status is not successful.

Error categories

Visible error	Meaning	Frontend response
Failed to fetch	Browser could not reach API or CORS blocked it.	Keep form state and show a clear error.
401	Token missing or expired.	Ask user to sign in again.
409	Duplicate username/ticker.	Display backend detail.
422	Invalid input.	Show validation message near action.
429	Rate limit reached.	Tell user to wait rather than retry rapidly.
5xx	Provider or server failure.	Show error without exposing secrets.

10. Evidence-first user experience

The interface is designed to be attractive without overstating what AI can know.

- Labels say AI-assisted or evidence-grounded instead of claiming guaranteed intelligence.
- A headline action is paired with risk, strengths, warnings, and the exact quantitative proof.
- Advanced details are collapsed by default so beginners are not overwhelmed.
- News is clickable and attributable instead of appearing as unsupported text.
- Backtest proof names adjusted prices, execution timing, transaction costs, and benchmark.
- Every research result includes a financial disclaimer.
- Empty states teach the next action instead of showing blank panels.

Trustworthy marketing

StockPilot can truthfully be described as AI-assisted because agents review evidence. It should not be described as an AI that predicts the market or guarantees profitable trades.

11. CSS design system and responsive behavior

globals.css defines reusable colors, spacing, cards, buttons, grids, transitions, and mobile breakpoints.

Token or pattern	Use
--ink / --muted	Primary and secondary text.
--green / --green-2	Brand, positive actions, links, focus.
--mint	Soft highlights and badges.
--amber / --red	HOLD warnings and negative states.
--line / --panel / --paper	Borders, cards, and page surfaces.
--shadow	Consistent elevation for important panels.
clamp()	Typography that scales with screen size.
CSS grid	Metric, signal, comparison, and saved-card layouts.

Responsive breakpoints

Below 900px, the fixed sidebar becomes a horizontal navigation bar, grids reduce columns, and the backtest form rearranges. Below 650px, result grids become single-column, metrics use two columns, and controls stack for touch screens.

Motion and accessibility

Short transitions clarify focus and new results. prefers-reduced-motion disables meaningful animation duration for users who request less motion. Inputs have visible focus rings, navigation reports aria-current, and semantic buttons and forms support keyboard use.

12. Local setup and environment variables

The frontend needs only the public API address. Start the backend first.

1. Clone the frontend repository and open it separately from the backend.
2. Install Node.js 22.13 or newer.
3. Run `npm install`.
4. Copy `.env.example` to `.env.local`.
5. Keep `NEXT_PUBLIC_API_URL=http://127.0.0.1:8000` for local development.
6. Run `npm run dev` and open the displayed localhost URL.

Commands

```
npm run dev # development server
npm run lint # ESLint checks
npm test # lint + production build
npm run build # Next.js production build
npm start # serve completed production build
```

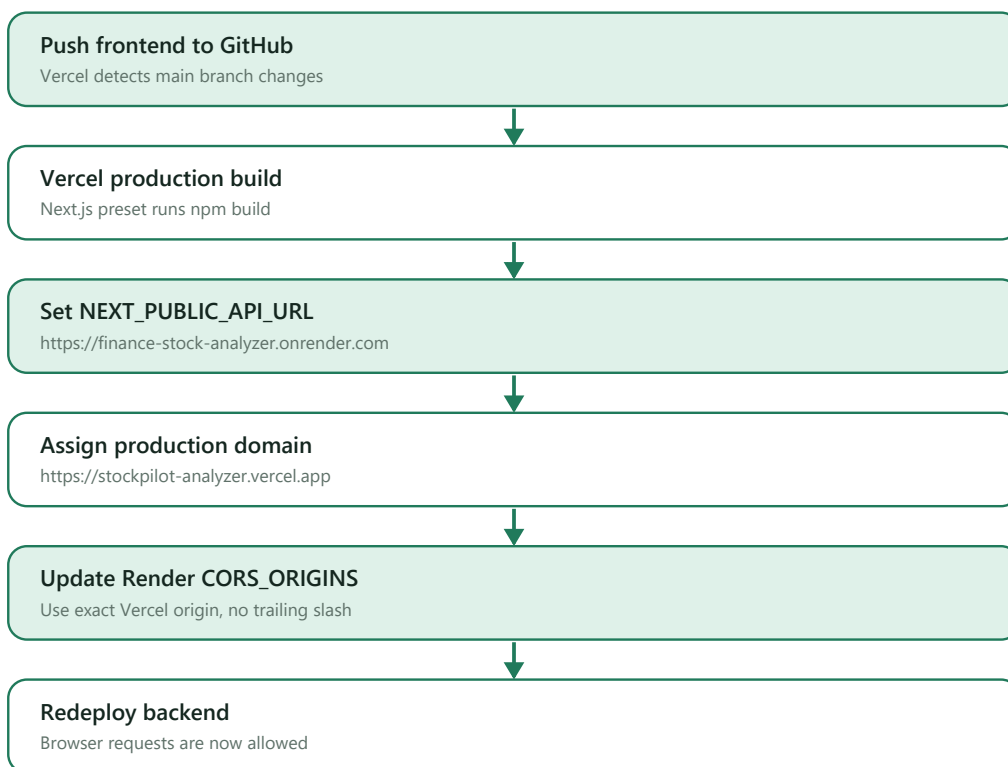
Public does not mean secret

Any variable beginning `NEXT_PUBLIC_` is embedded in browser code. Store only the API address there. Backend secrets must remain on Render.

13. Vercel deployment and CORS

Vercel builds the frontend from GitHub. Render must explicitly trust the final browser origin.

The frontend URL and API URL point in opposite directions.



CORS explained simply

Browsers block JavaScript from calling a different origin unless the API gives permission. CORS_ORIGINS is the backend's allowlist of trusted frontend origins. Changing a Vercel domain requires updating that allowlist.

Setting	Where	Value
NEXT_PUBLIC_API_URL	Vercel	https://finance-stock-analyzer.onrender.com
CORS_ORIGINS	Render	https://stockpilot-analyzer.vercel.app
DATABASE_URL	Render only	Private Neon connection string
GEMINI_API_KEY	Render only	Private provider key

14. Testing and troubleshooting

Frontend validation should catch type, lint, and production-build problems before Vercel does.

Run before pushing

```
npm run lint  
npm test
```

Symptom	Likely cause	Fix
Vercel 404	Wrong framework/build configuration.	Use Next.js preset and standard npm build.
Failed to fetch	API URL or CORS mismatch.	Verify both deployment variables and redeploy.
Port 3000 in use	Another dev server is running.	Use the new displayed port or stop old process.
Watchlist empty	Nothing saved for current production user.	Analyze and click + Watchlist.
News empty	Backend/provider returned no attributable articles.	Run a fresh analysis after backend deploy.
Times appear wrong	UTC text used directly.	Format ISO timestamp with Date.toLocaleString.
Login stops after refresh	Expired/invalid token.	Clear session token and sign in again.

15. Frontend interview questions

Explain behavior in terms of data flow and tradeoffs, not only library names.

Q1. Explain the frontend architecture.

It is a separate Next.js client application deployed on Vercel. A typed API helper calls the Render FastAPI backend. React state manages authentication, tabs, jobs, results, personal data, and errors. The browser contains no provider or database secrets.

Q2. Why Next.js if this is mainly one page?

It provides a production React toolchain, optimized fonts and metadata, a reliable build, and first-class Vercel deployment. For this interactive dashboard, the main page is a client component.

Q3. Why use TypeScript?

API responses are large and nested. Types document the contract, improve autocomplete, and catch wrong field usage during development and build. Runtime status and parsing checks are still necessary.

Q4. How does authentication work in the UI?

The form requests a JWT from `/auth/token`, stores it in `sessionStorage`, attaches it as a Bearer token, calls `/auth/me`, and then loads personal data. Sign-out removes the token and clears user state.

Q5. Why use polling for analysis?

Agent analysis is slower than a typical request. Polling lets the API return a job ID quickly and lets the UI show queued and running states. It is simpler than WebSockets for a small demo.

Q6. How do you keep the UI understandable despite technical output?

The layout uses progressive disclosure: action and risk first, plain-English evidence next, and metrics, proof, news, and raw details in expandable sections. Empty and error states explain the next step.

Frontend interview questions - continued

Q7. What is CORS?

It is a browser security policy for cross-origin requests. The frontend and backend have different domains, so Render must return headers allowing the exact Vercel origin.

Q8. Why is NEXT_PUBLIC_API_URL safe?

It only identifies the public API server. NEXT_PUBLIC values are visible in browser bundles, so secrets such as Gemini keys, JWT signing secrets, and database URLs must never use that prefix.

Q9. How is responsive design implemented?

CSS grids and flexible widths form the desktop layout. Media queries at 900px and 650px convert the sidebar, reduce grid columns, stack forms and cards, and simplify labels for smaller screens.

Q10. What accessibility choices exist?

Inputs have visible focus states, navigation uses aria-current, controls use semantic buttons and forms, color is paired with text labels, and reduced-motion preferences minimize animation.

Q11. How are API errors handled?

The shared helper parses JSON when possible, handles 204 responses, and throws backend detail for failed statuses. Event handlers put those messages into dedicated error state without destroying existing form input.

Q12. Why store saved analysis as JSON?

The UI can reopen the exact result shown at that point in time without recomputation. The tradeoff is weaker relational querying and the need to handle older response shapes carefully.

Q13. What would you refactor as the frontend grows?

Split API access into a service, authentication into a context or hook, analysis/backtest sections into components, and use a server-state library for caching and request lifecycle management.

Frontend interview questions - deeper follow-ups

Q14. Why sessionStorage instead of localStorage?

sessionStorage limits persistence to the tab session, reducing how long a token remains. Both are accessible to JavaScript and therefore vulnerable to XSS; secure HTTP-only cookies would be stronger for a hardened app.

Q15. What triggers a React re-render?

Updates through state setters such as setAnalysis or setTab. useRef can store polling control without triggering a render, and useMemo/useCallback avoid unnecessary derived work or function changes.

Q16. How do you avoid exaggerating AI?

The UI states that agents interpret verified evidence and that signals are reproducible. It shows calculation proof and sources and avoids labels such as guaranteed prediction or AI accuracy.

Q17. Why separate frontend and backend repositories?

They have different runtimes, secrets, deployment platforms, and release cycles. Separation keeps the browser bundle independent and prevents backend environment files from being mixed into frontend deployment.

Q18. What does npm test verify?

In this project it runs ESLint and a production Next.js build. That catches static code issues, type/build errors, and deployment configuration problems; it is not yet a full browser interaction suite.

Q19. How would you test the UI further?

Add unit tests for formatting and API helpers, component tests for states, and Playwright end-to-end tests covering registration, analysis polling, watchlist, saved research, and backtesting with a controlled test API.

Q20. What is the strongest UX decision?

A good answer is progressive disclosure with proof: beginners see a concise result, while professionals can inspect the underlying metrics, formulas, rule contributions, sources, and methodology.

16. Limitations and sensible improvements

The current frontend is appropriate for a portfolio demo; improvements should solve a measured need.

Current limitation	Improvement if needed
One large page component	Split into feature components and hooks.
Manual polling	Use a reusable polling hook or server-state library.
Token in sessionStorage	Move to secure HTTP-only cookies for hardened authentication.
Build validation only	Add component and Playwright end-to-end tests.
No chart library	Add an equity curve only if it improves comprehension.
No service worker/offline mode	Add only if offline saved research becomes a requirement.

Final deployment checklist

- Live frontend opens at the final Vercel domain.
- Backend /health reports API and database healthy.
- Registration, login, analysis, backtest, watchlist, saved research, and sign-out work.
- Refresh preserves the current tab token session and personal data reloads.
- No secret appears in GitHub or Vercel public variables.
- README links and screenshots point to the final production URLs.

Final revision sentence

The StockPilot frontend is a typed Next.js interface that manages authenticated API calls, polls long-running analysis jobs, and turns complex financial JSON into a responsive, evidence-first user experience.

Live application: <https://stockpilot-analyzer.vercel.app>

Frontend repository: <https://github.com/mayank2OP/finance-stock-analyzer-frontend>

Backend API: <https://finance-stock-analyzer.onrender.com>